



Distributed Microservice Architecture with Docker Swarm

Smart Tracking System

Group: 4. Group - Container & Container Management

Prepared by:

Hızır Çiçekdağ (212010020079)

Tuğba Köroğlu(212010020110)



What is Docker and Where is it Used?



What is Docker?

In the simplest terms, Docker is a technology that takes your code along with all required libraries, tools, and system configurations needed for it to run, and packages them into an isolated, standardized unit called a "container." Your application runs inside this container in its own environment, completely independent from the external system.

Where is it used?



To Solve the "It Worked on My Computer"

Problem: Your code runs exactly the same on your Mac, your friend's Windows machine, or a live cloud server. This is because the application is shipped together with its own environment inside a container, allowing it to run consistently everywhere.



In Microservice Architectures

As in our project, it is used to run MongoDB, RabbitMQ, and Node.js services side by side on the same system without interfering with each other's configurations or versions, ensuring smooth and isolated operation of each service.



Fast Installation and Deployment (Deploy):

Instead of spending hours setting up a new server (installing Node.js, MongoDB, and configuring everything), Docker is used to simply transfer the application's Docker image to the server and bring it up and running within seconds.



Resource Efficiency:

Compared to traditional and heavy Virtual Machines, Docker is much more lightweight. Since it does not replicate the entire operating system, it places less load on the system. This makes it ideal for running dozens of containers simultaneously and efficiently on the same machine.



Its Role in Our Project:

We used Docker to fully isolate our three developed Node.js microservices, the Web Dashboard, RabbitMQ, and three separate MongoDB instances from each other.

What is Docker Swarm and Where is it Used?



What is Docker Swarm?

In its shortest form: Docker Swarm is an "**Orchestration (Container Management)**" tool that allows you to automatically manage dozens or hundreds of Docker containers from a single center.

If Docker is the "**worker**" who packages your application, Docker Swarm is the "**General Manager**" who maintains the order of these workers, establishes communication between them, and decides who works where.



Against System Crashes (Self-Healing):

When a microservice throws an error and shuts down in the production environment (live), Swarm notices this even if you are not at the computer. It pulls the plug on the stopped container and instantly opens a new one in its place, preventing the system from being interrupted.



When Excessive Traffic Arrives (Load Balancing):

When thousands of users suddenly enter the system, Swarm does not dump this massive load (traffic) onto a single container; it distributes it equally and fairly among the running container replicas.



To Start Complex Systems With a Single Button:

Manually launching 3 databases, 1 message queue, and 3 microservices one by one is challenging, as in our project. Swarm is used to read all this architecture through a single file (docker-compose.yml) and launch it with a single command.



In Secure Internal Network Setup:

Used when creating encrypted private networks (overlay networks) where containers are isolated from the external world and can only securely communicate with each other.

Its Place in Our Project: We used it to manage the entire system architecture from a single center (Manager Node) using the overlay network driver.

Where is it Used? Docker alone cannot automatically manage containers. Swarm is used when dozens of containers need to be launched, hidden networks established between them, Load Balancing performed, and crashed containers automatically brought back up (Self-Healing) is required.

Application in the Code: Dockerfile



Where is it Located?

It is located separately inside each microservice folder.

Paths:

`/package-service/Dockerfile`

`/tracking-service/Dockerfile`

`/notification-service/Dockerfile`



What Does it Do Inside the Code?

FROM node:18-alpine

-> Pulls the lightweight Node.js base image where the service will run.

COPY package*.json ./ ve **RUN npm install**

-> Installs the libraries inside the container.

COPY . .

-> Copies the JavaScript code we wrote into the container.

EXPOSE [Port]

-> Specifies the port (3001, 3002, 3003) through which the service will communicate with the outside world.

Application in the Code: docker-compose.yml



Where is it Located

It is located in the project's top most root directory.



How Was Swarm Orchestration Provided in the Code?

depends_on -> - **rabbitmq** -> Brought to a flat list format compatible with Swarm (Conditions were removed).

image: tags were added, enabling Swarm to recognize the images by their names.

networks -> **driver: bridge** -> The structure we used without Swarm

we need to change this part as follows when transitioning to the Swarm structure

networks -> **driver: overlay** -> The virtual network driver that Swarm uses to connect services was coded.

Why "Bridge" Instead of "Overlay"?



Bridge Network

It connects containers running on only a single physical machine (for example, only your computer) to each other.

It is the default network of Standard Docker. It cannot communicate with a container on another computer.



Overlay (Layer/Cover) Network

It is the heart of Docker Swarm. Swarm's main purpose is to turn multiple computers (servers) into a single küme (cluster).

The overlay network is a magical and encrypted virtual network that connects containers running on different physical computers to each other as if they were in the same room, on the same network.

Application in the Code: .env and Service Communication

Why Was localhost Canceled?



Container Isolation:

Every Docker container has its own, externally closed `localhost` (127.0.0.1).



Connection Problem:

When a microservice makes a request to localhost, it looks inside itself, cannot find the other services (MongoDB, RabbitMQ), and throws an error.



Solution: Swarm Internal DNS System:

The `localhost` words in the connection addresses were replaced with the service names inside `docker-compose`.



Mechanism:

Swarm provides uninterrupted communication by dynamically converting service names to the correct IP addresses in the background.

Old: `mongodb://localhost:27017`

New: `mongodb://mongo_package:27017`

Conclusion



Centralized Orchestration:

All those loose containers turn into orderly "Service" networks managed by Swarm. The entire system is managed with a single command (`stack deploy`).



Self-Healing

Swarm monitors the system second by second. It keeps the system running by instantly creating a new one in place of a closed or crashed service, even while we sleep.



Smart Network (`overlay`) and Load Balancing:

Thanks to Internal DNS, services find each other without asking for an IP (e.g.: `mongo_package`) and, in moments of extreme intensity, Swarm fairly distributes the incoming traffic among the containers.